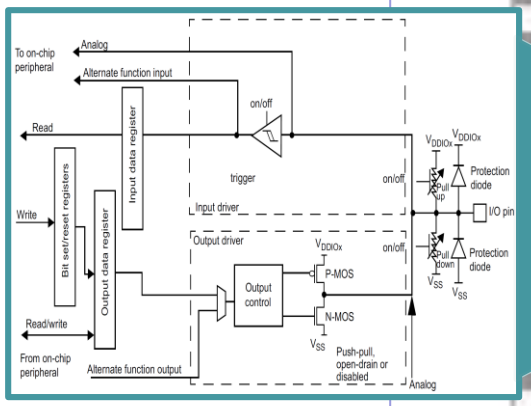
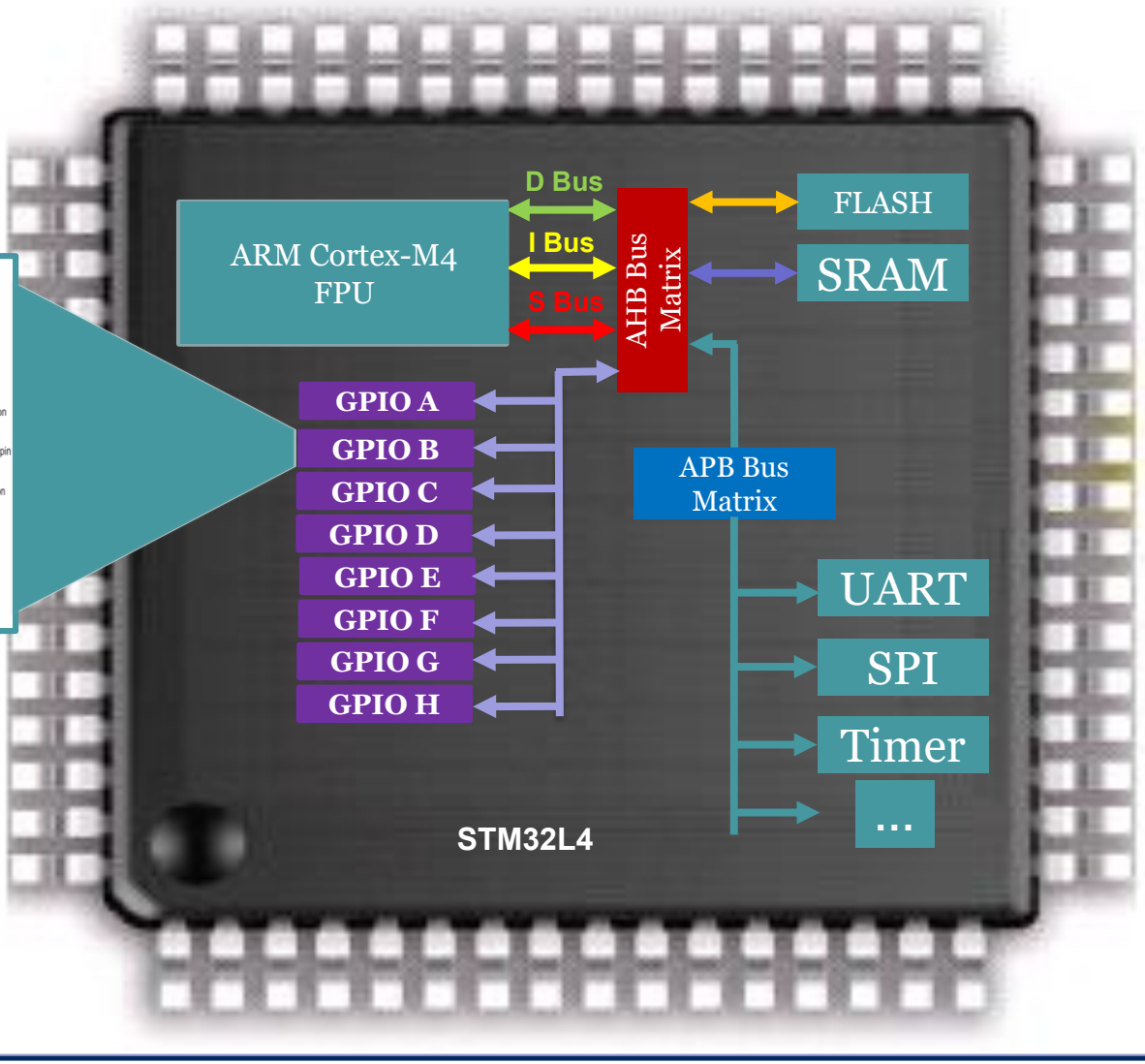
ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute

I/O Peripherals

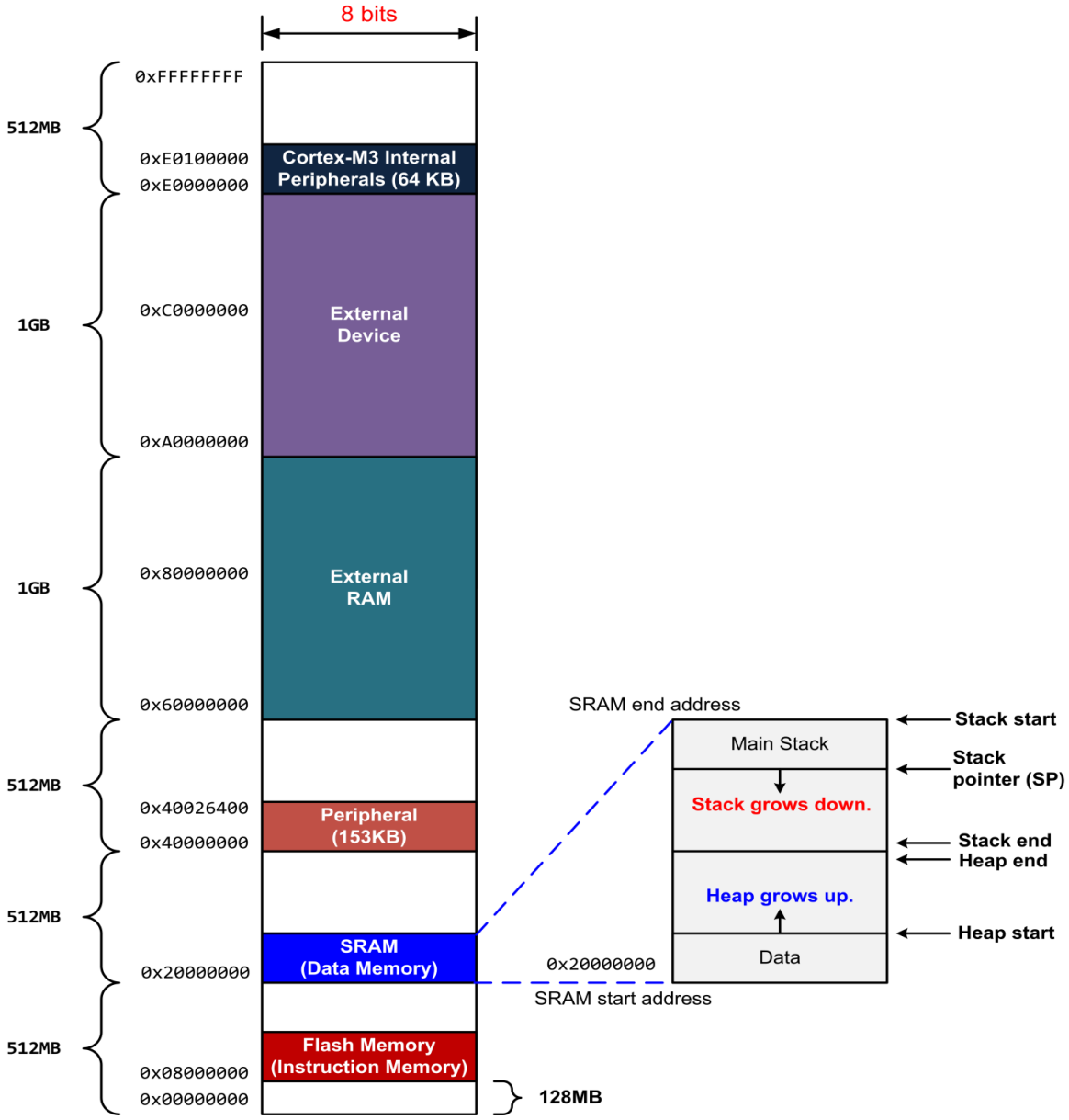




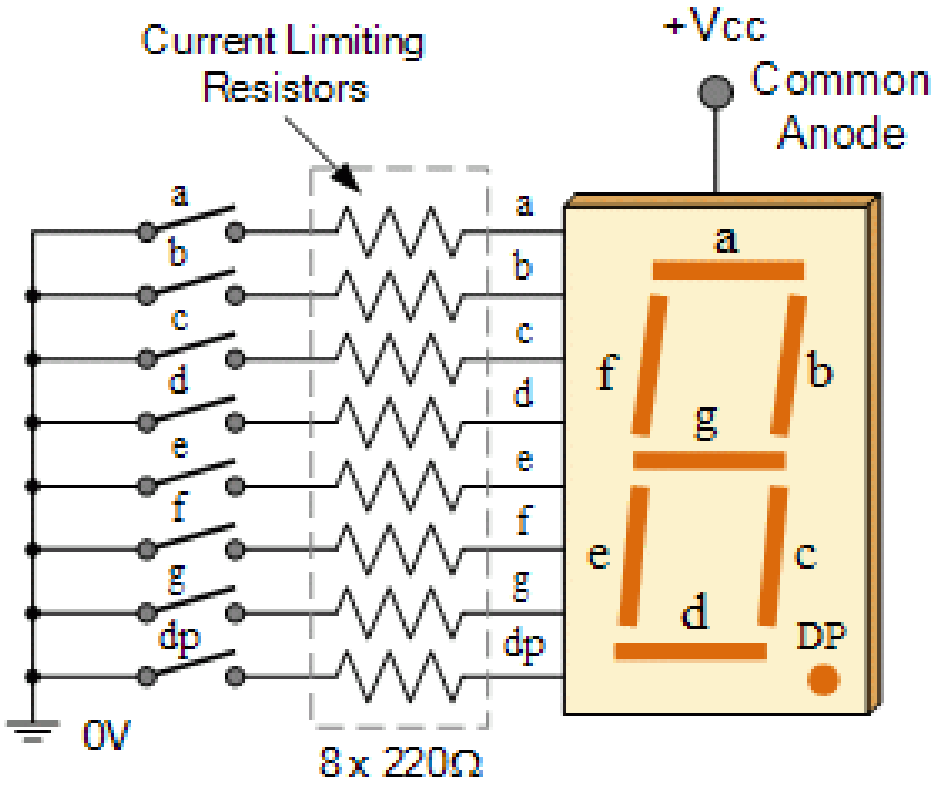
GPIO Introduction

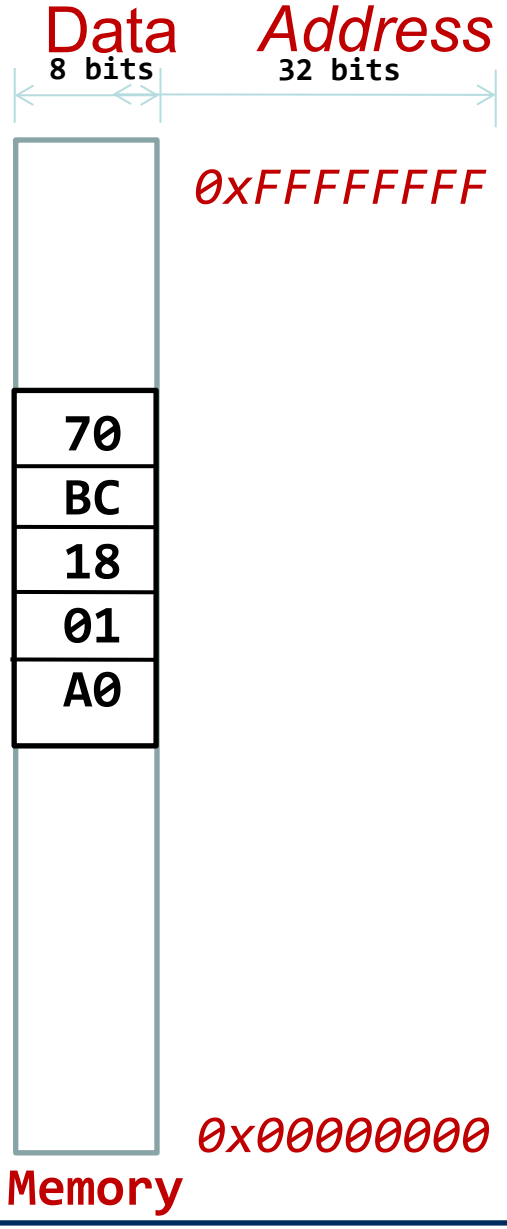
- GPIO means General Purpose Input/Output
- GPIO translates digital data inside the processor to analog voltages in the real world
- Digital 1 = analog 3.3V, digital 0 = analog 0V (and vice versa)

Memory Map


8 bits
0xFFFFFFFF
512MB
0xE0100000
0xE0000000
Cortex-M3 Internal Peripherals (64 KB)
1GB
0xC0000000
External Device
0xA0000000
1GB
0x80000000
External RAM
0x60000000
512MB
0x40026400
Peripheral (153KB)
0x40000000
512MB
SRAM (Data Memory)
0x20000000
512MB
Flash Memory (Instruction Memory)
0x08000000
0x00000000
128MB
SRAM end address
Main Stack
Stack grows down.
Heap grows up.
Data
0x20000000
SRAM start address
Stack start
Stack pointer (SP)
Stack end
Heap end
Heap start

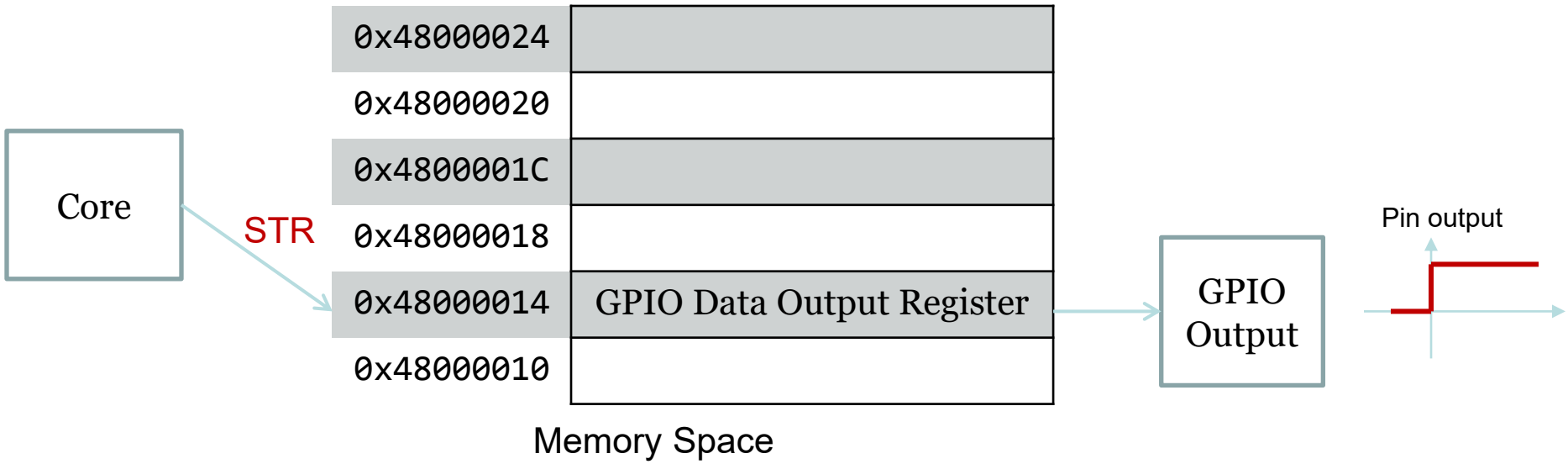
7-Segment Display





Interfacing Peripherals

- **Memory-mapped I/O**
  - Each device registers is assigned to a memory address in the address space of the microprocessor
  - Use native CPU load/store instructions: `LDR/STR Reg, [Reg, #imm]`

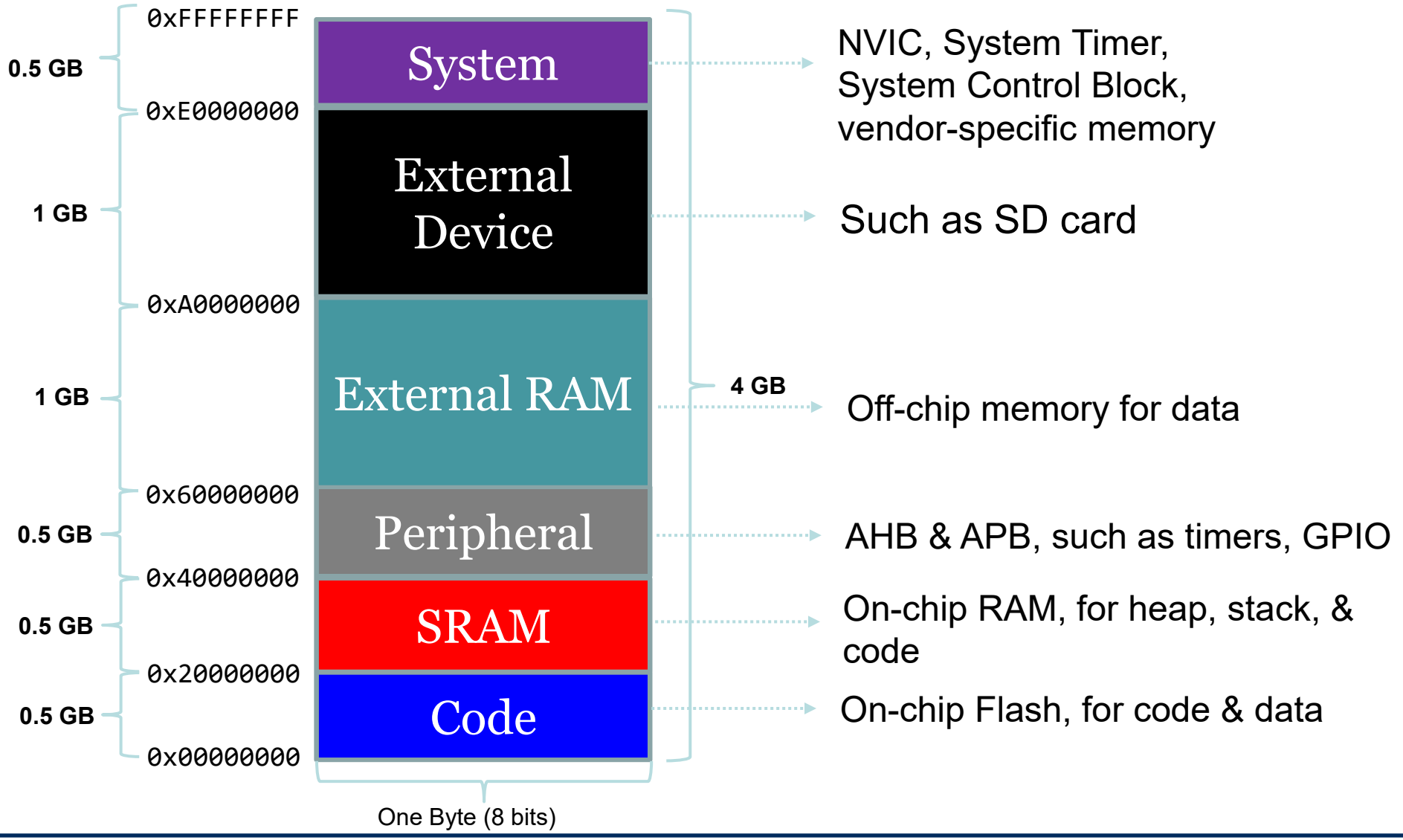


ARM Cortex-M microprocessors use memory-mapped I/O.

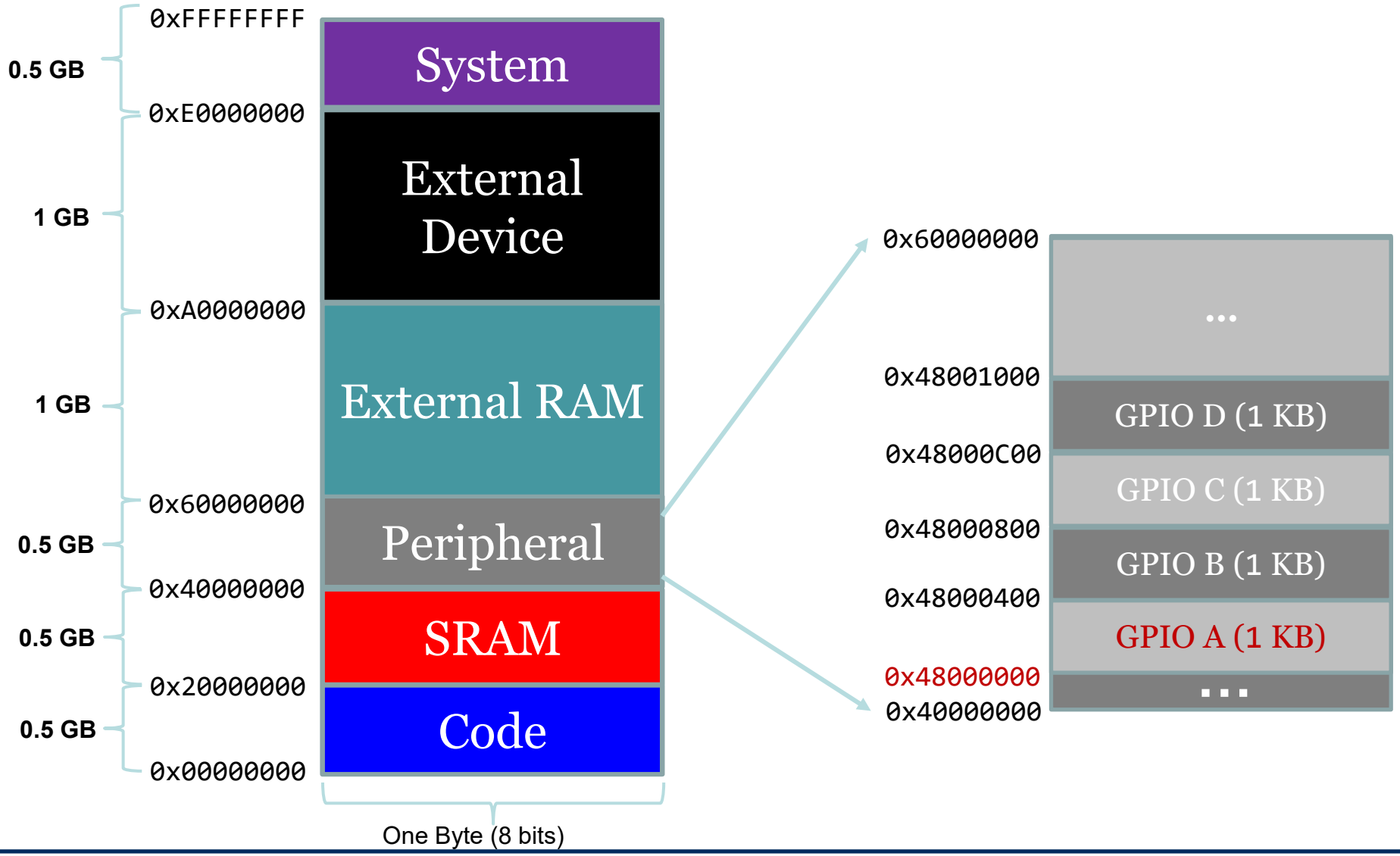
GPIO Introduction

- Memory-Mapped I/O:
  - Digital “bits” in “registers” correspond to analog voltages on physical pins
  - Output pin: By setting a bit in this register to a 1, you produce 3.3 V at the pin
  - Input pin: By applying >2V at the pin, you produce a digital 1 in the register

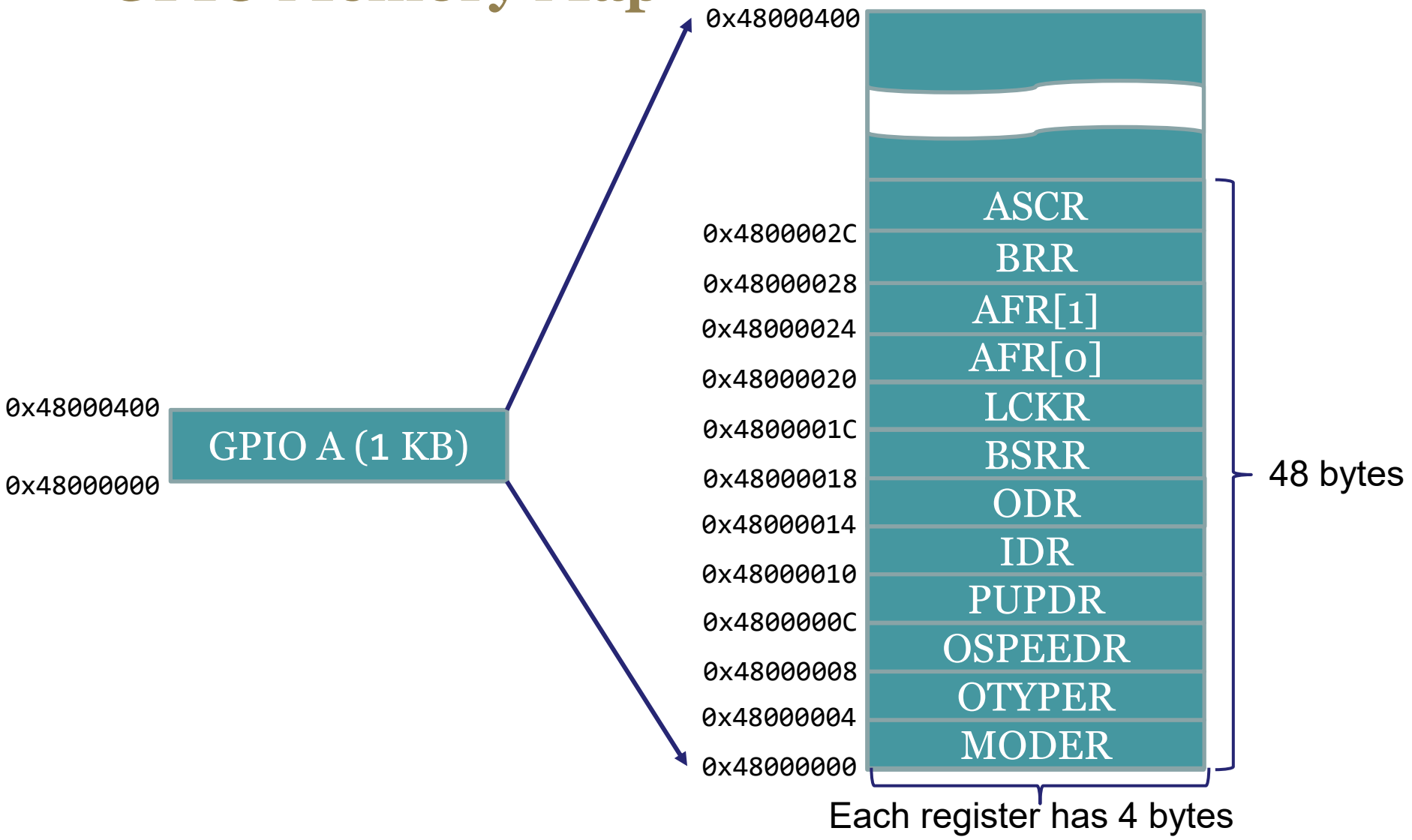
Memory Map of Cortex-M4



Memory Map of STM32L4

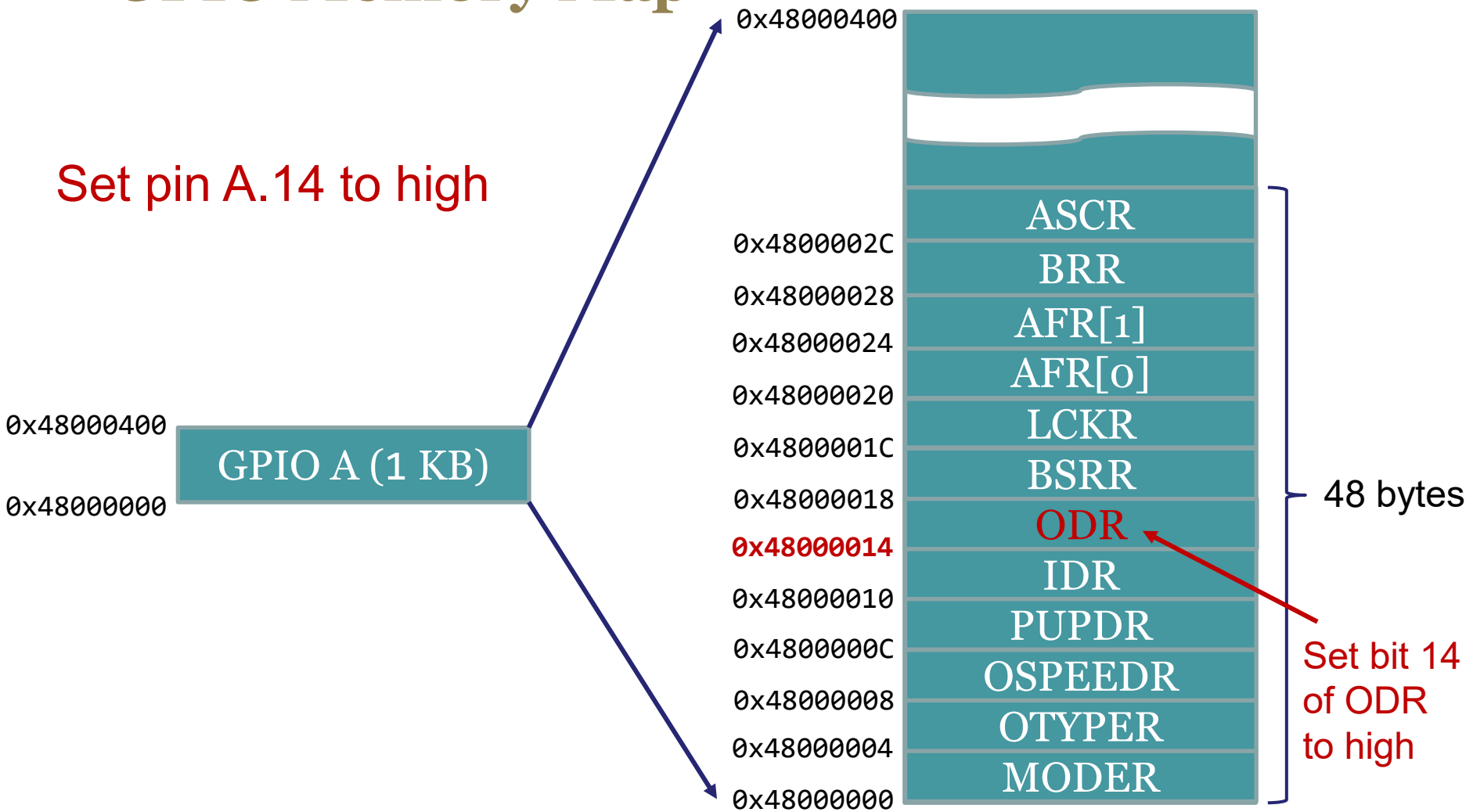


GPIO Memory Map

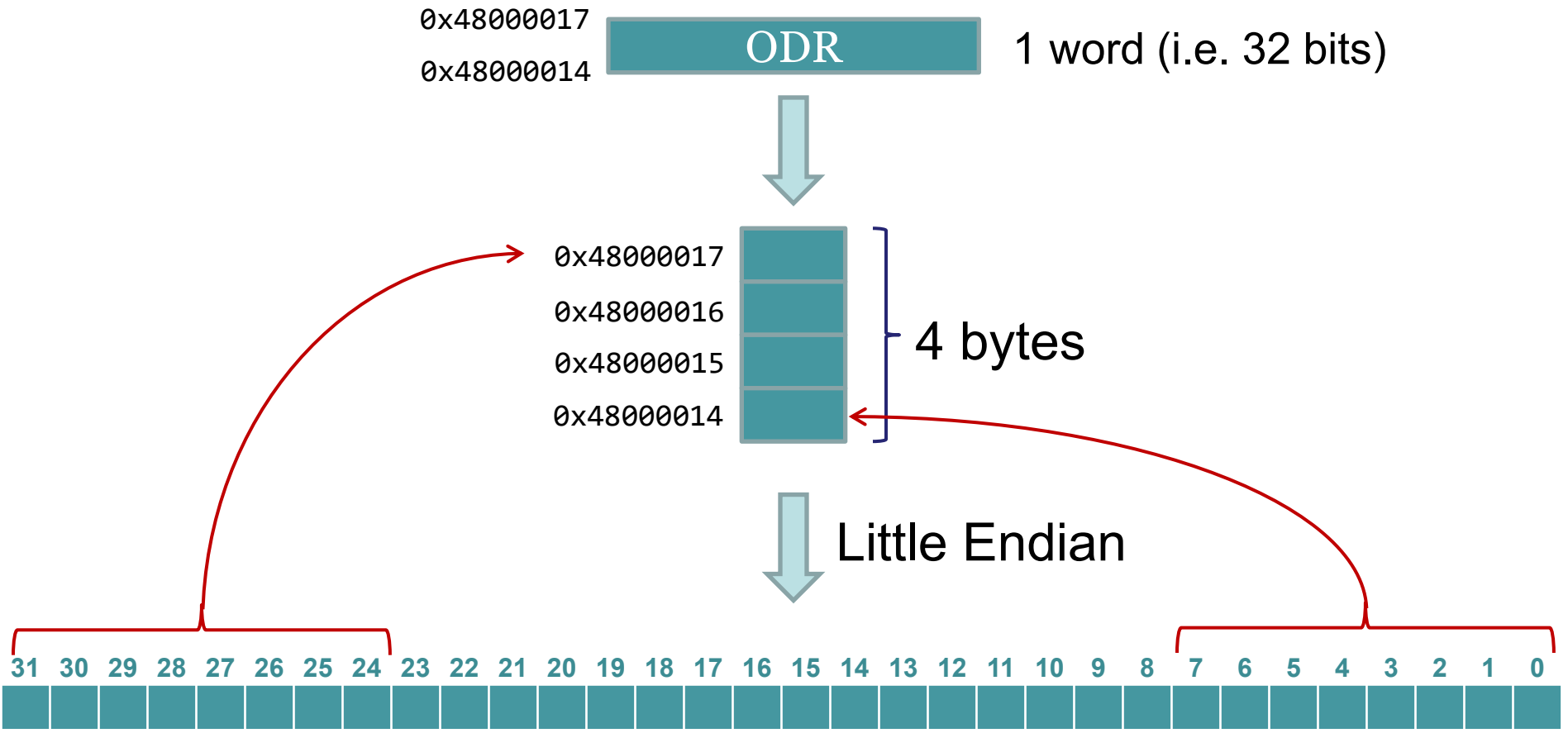


GPIO Memory Map

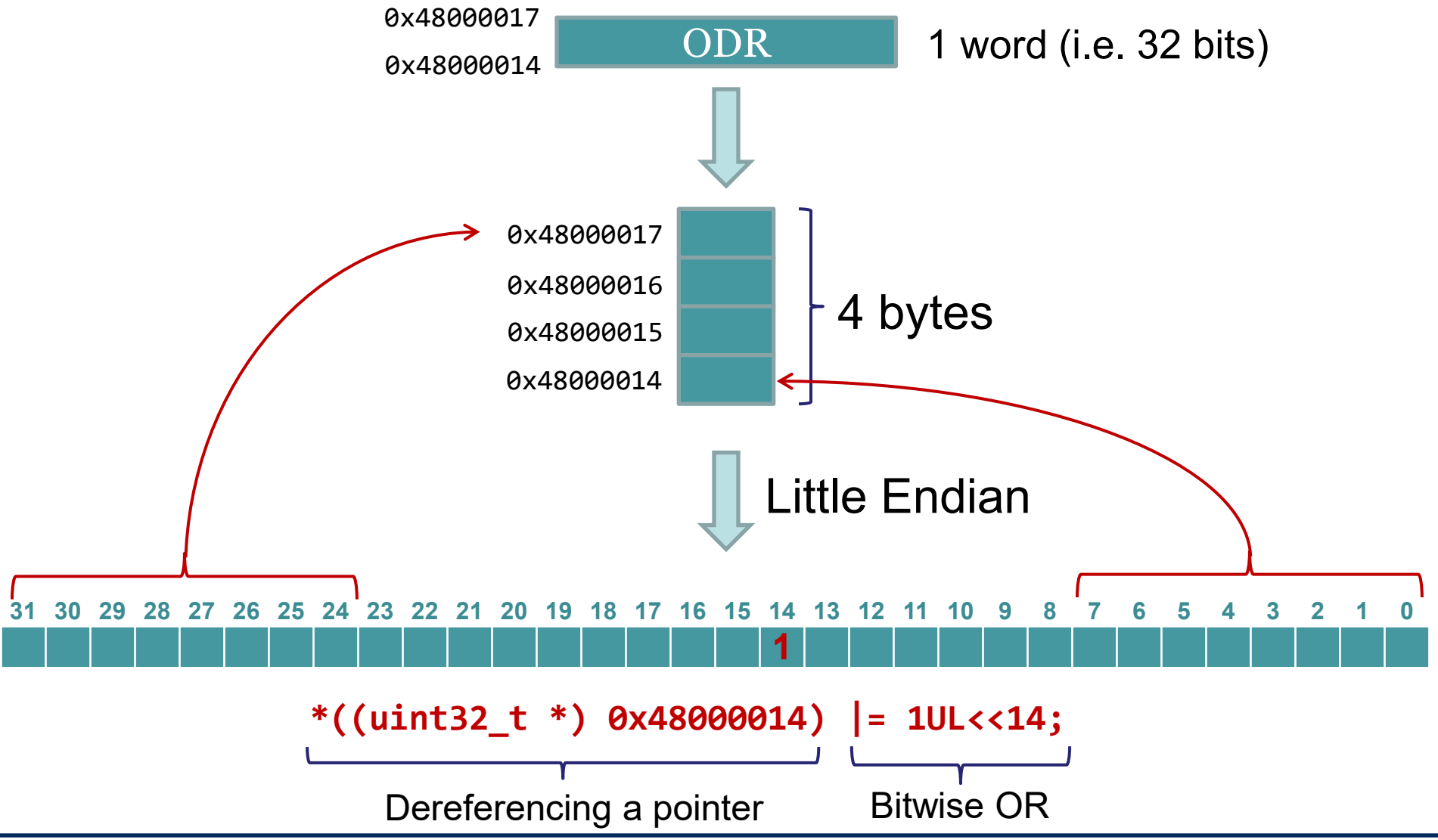
Set pin A.14 to high



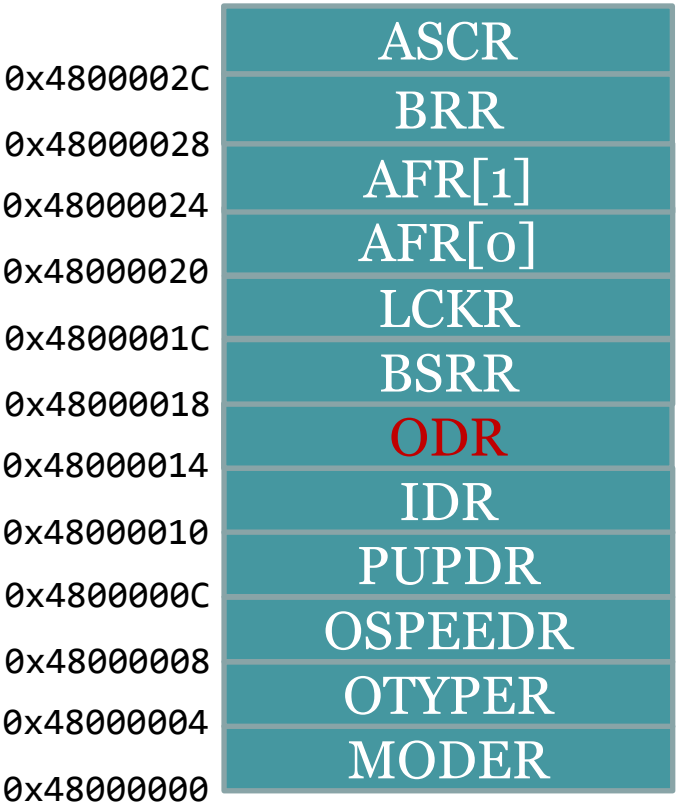
Output Data Register (ODR)



Output Data Register (ODR)



Dereferencing a Memory Address




```
typedef struct {
  volatile uint32_t MODER;    // Mode register
  volatile uint32_t OTYPER;   // Output type register
  volatile uint32_t OSPEEDR;  // Output speed register
  volatile uint32_t PUPDR;    // Pull-up/pull-down register
  volatile uint32_t IDR;      // Input data register
  volatile uint32_t ODR;      // Output data register
  volatile uint32_t BSRR;     // Bit set/reset register
  volatile uint32_t LCKR;     // Configuration lock register
  volatile uint32_t AFR[2];   // Alternate function registers
  volatile uint32_t BRR;      // Bit Reset register
  volatile uint32_t ASCR;     // Analog switch control register
} GPIO_TypeDef;

// Casting memory address to a pointer
#define GPIOA ((GPIO_TypeDef *) 0x48000000)
```

```
GPIOA->ODR |= 1UL<<14;
```

or

```
(*GPIOA).ODR |= 1UL<<14;
```

Basic C instructions

- {Module}->{Register} gives access to a register

Basic C Instructions

- Set bit N in a register (N = 0,1,2,…)

- MOD->REG |= (1UL << N);

Basic C Instructions

- Clear bit N in a register (N = 0,1,2,…)

- MOD->REG &= ~(1UL << N);

Basic C Instructions

- Toggle bit N in a register

- MOD->REG ^= (1UL << N);

Bit Mask

- A bit mask is a binary number where:
  - All the bits you want to change (either set, reset, or toggle) are set to 1
  - All the bits that you do not want to change are set to a 0

Example Bit Mask 1

- Bits start at bit 0
- Set bits 1, 2, and 4
- Mask = bits 1, 2, and 4 are set to 1

…	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	1	0

- Mask = 0x00000016

Example Bit Mask 2

- Clear bits 0, 3, 5, 9

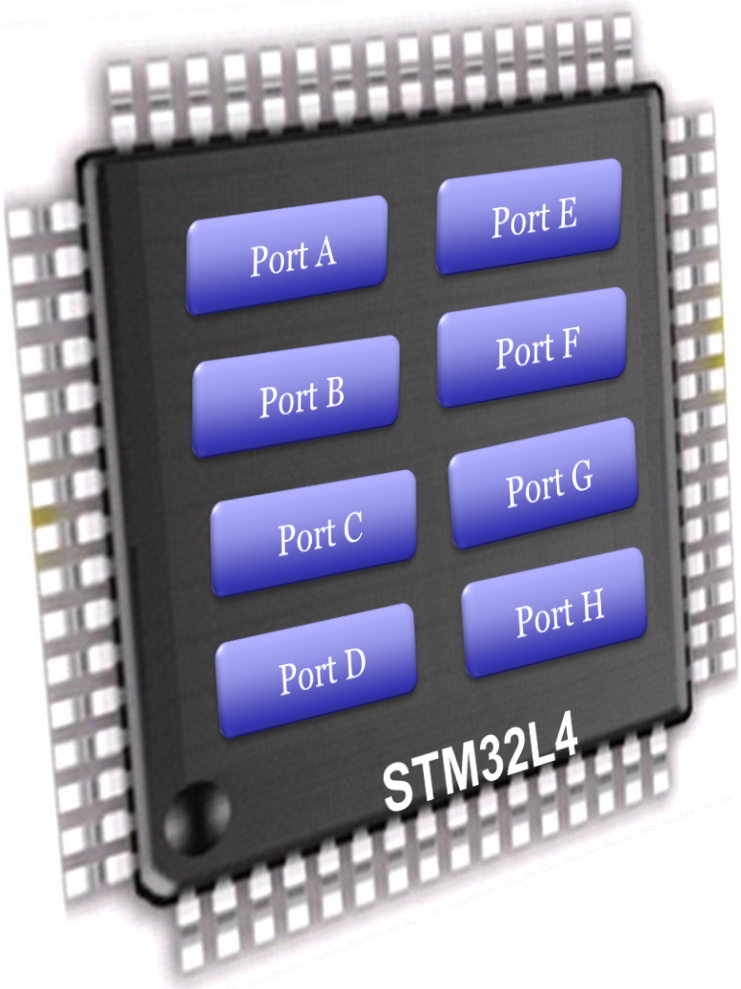
...	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	0	0	1

- Mask = 0x00000229

Using a bit mask

- Setting bits:
  - {MOD}->{REG} |= {MASK};
- Clearing bits:
  - {MOD}->{REG} &= ~{MASK};
- Toggling bits:
  - {MOD}->{REG} ^= {MASK};

General Purpose Input/Output (GPIO)

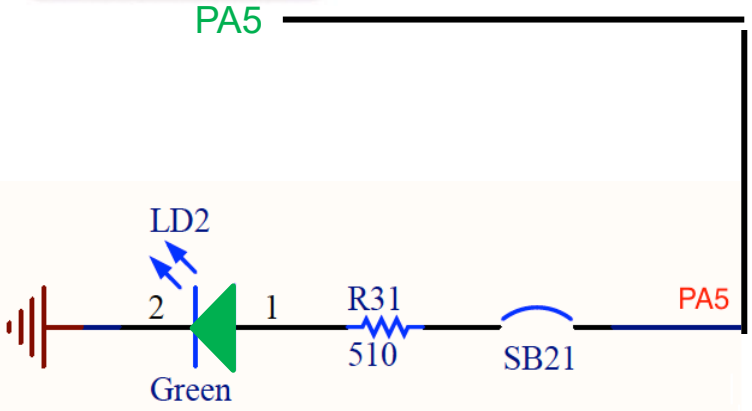


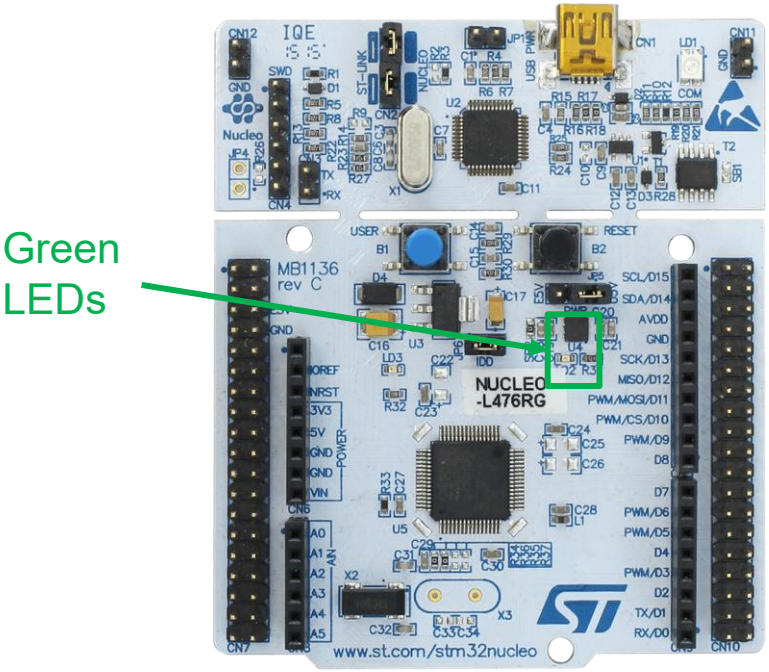
- 8 GPIO Ports:

  A, B, C, D, E, F, G, H

- Up to 16 pins in each port

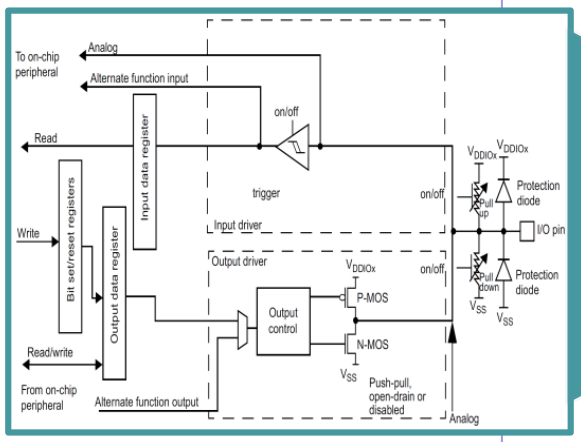
Green LED (PA5)

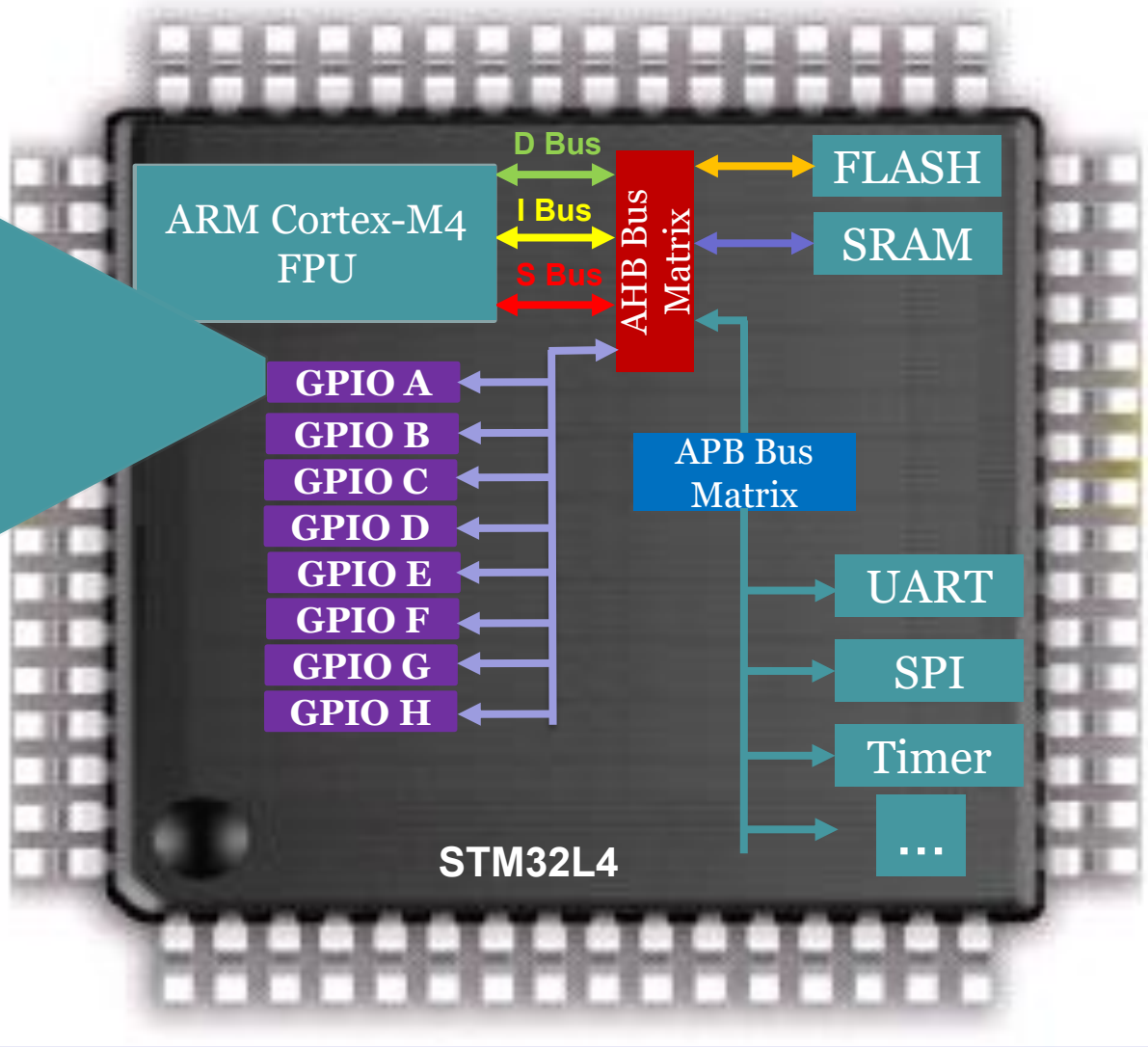




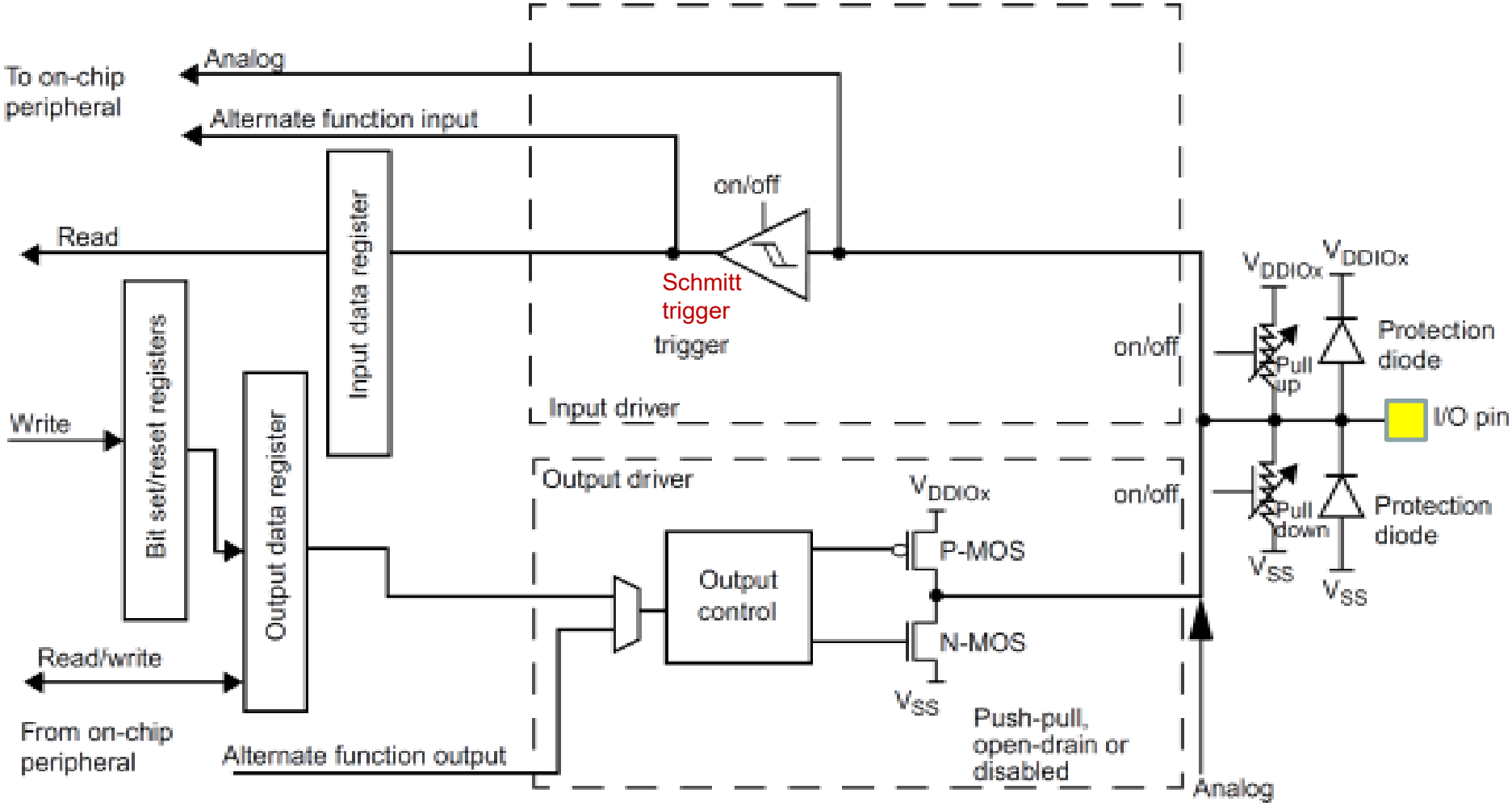
PA5	Green LED
High	On
Low	Off

General Purpose Input/Output (GPIO)





Basic Structure of an I/O Port Bit
Input and Output

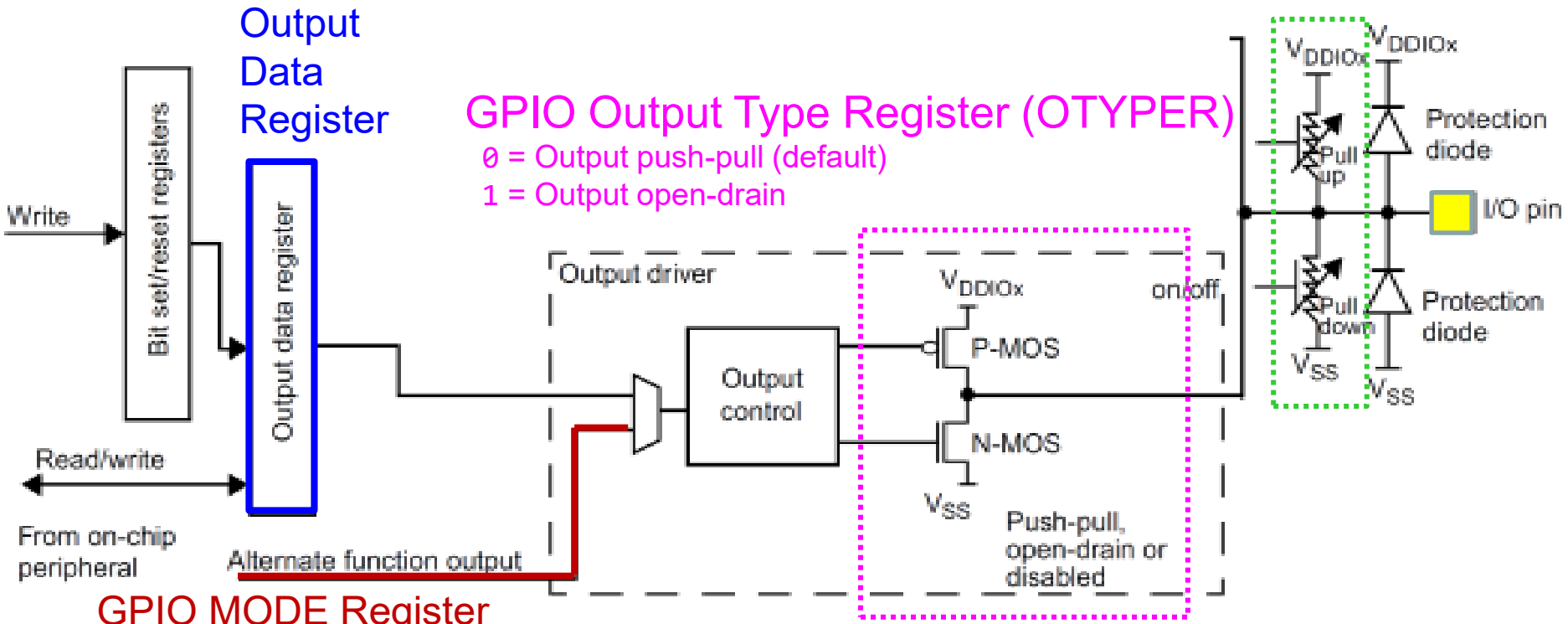


Basic Structure of an I/O Port Bit: Output

GPIO Pull-up/Pull-down Register (PUPDR)

00 = No pull-up, pull-down 01 = Pull-up

10 = Pull-down 11 = Reserved



GPIO MODE Register

00 = Input, 01 = Output,

10 = AF, 11 = Analog (default)

How to use GPIO

1. Turn on GPIO module clock in Reset and Clock Control (RCC).
2. Set the mode of the GPIO pin in the GPIO Mode Register (MODER).
3. Set Output Type and Pull-Up Configuration.
4. If output, write data to Output Data Register (ODR). If input, read data from Input Data Register (IDR).

GPIO Initialization

- Turn on the clock to the GPIO Port (e.g. Port A)

```
RCC->AHBENR |= RCC_AHB2ENR_GPIOAEN; Reset and Clock Control (RCC)
```

- Configure GPIO mode, output type, speed, pull-up/pull-down

```
typedef struct
{
  __IO uint32_t MODER;
  __IO uint16_t OTYPER;
  uint16_t RESERVED0;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint16_t IDR;
  uint16_t RESERVED1;
  __IO uint16_t ODR;
  uint16_t RESERVED2;
  __IO uint16_t BSRRL; /* BSRR register is split to 2 * 16-bit fields BSRRL */
  __IO uint16_t BSRRH; /* BSRR register is split to 2 * 16-bit fields BSRRH */
  __IO uint32_t LCKR;
  __IO uint32_t AFR[2];
} GPIO_TypeDef;
#define PERIPH_BASE           ((uint32_t)0x40000000)
#define AHBPERIPH_BASE        (PERIPH_BASE + 0x20000)
#define GPIOA_BASE            (AHBPERIPH_BASE + 0x0400)
#define GPIOA                 ((GPIO_TypeDef *) GPIOA_BASE)
```

Enable Clock

- AHB2 peripheral clock enable register (RCC_AHB2ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	Res.	AESEN
													rw		rw

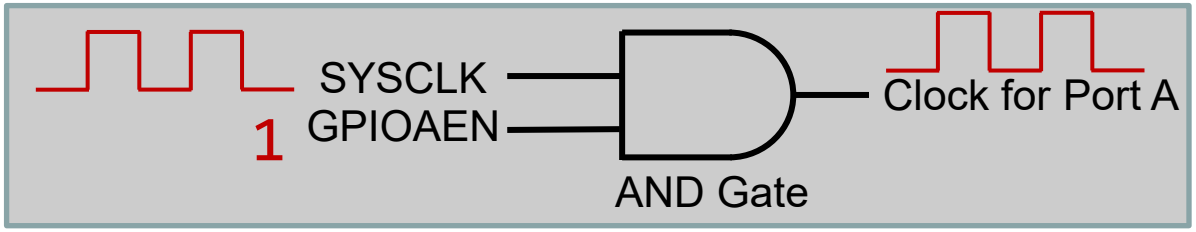
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADCEN	OTGFS EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **GPIOAEN:** IO port A clock enable

Set and cleared by software.

0: IO port A clock disabled

1: IO port A clock enabled




```
#define  RCC_AHB2ENR_GPIOAEN  ((uint32_t)0x00000001U) 00000001B

RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
```

GPIO Mode Register (MODER)

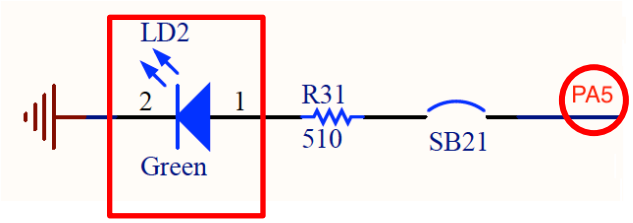
- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 5 Pin 1 Pin 0

Bits 2y+1:2y **MODEy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)




```
GPIOA->MODER &= ~(3UL<<10);  // Clear bits 10 and 11 for Pin 5
GPIOA->MODER |=   1UL<<10;    // Set bit 10, set Pin 5 as output
```

GPIO Output Type Register (OTYPE)

- 16 bits reserved, 16 data bits, 1 bit for each pin

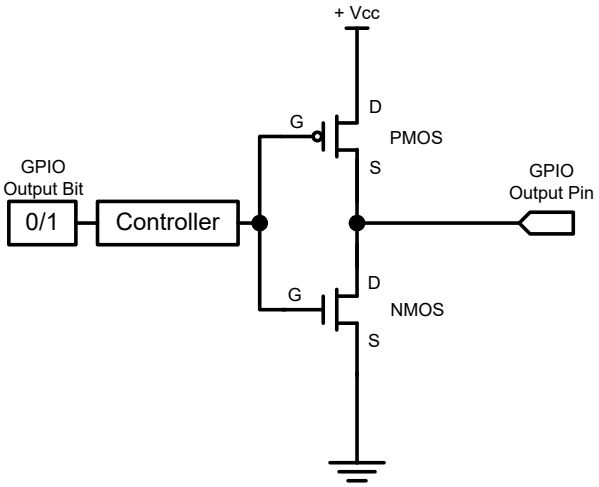
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **OTy:** Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output type.

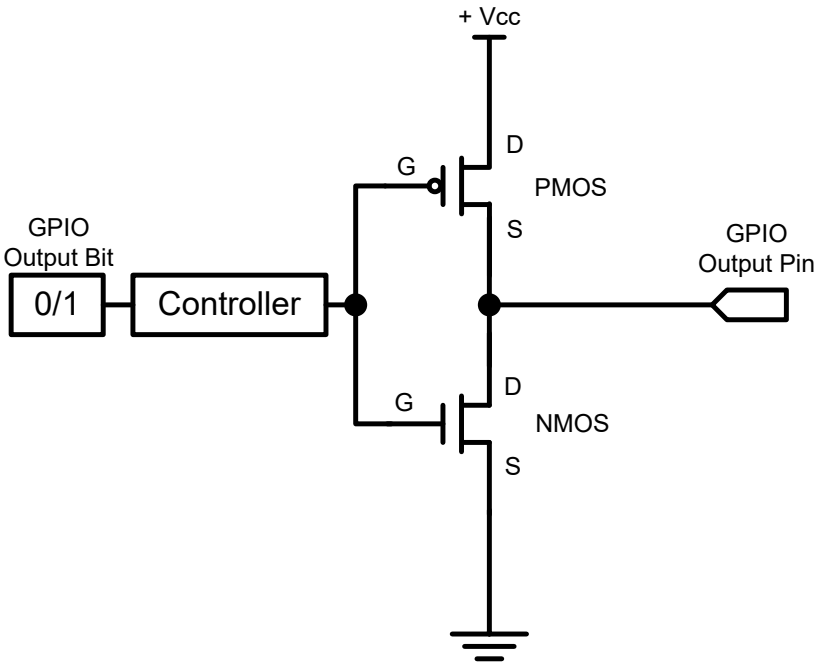
0: Output push-pull (reset state)

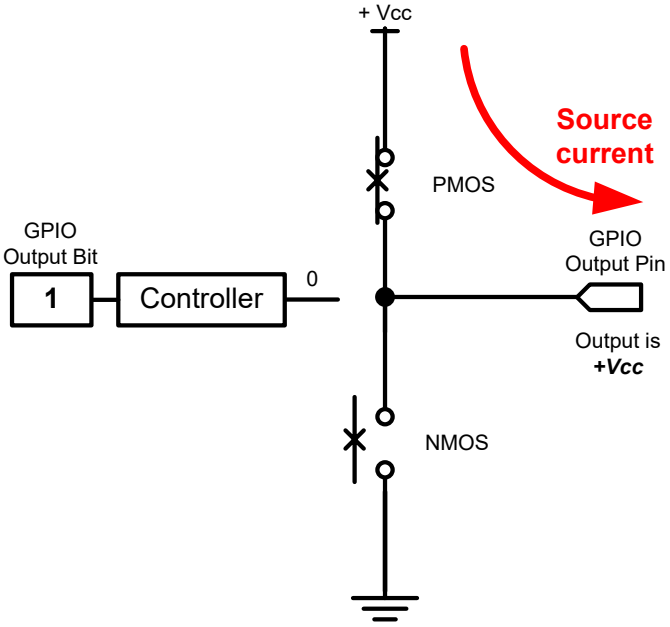
1: Output open-drain




```
GPIOA->OTYPE &= ~(1UL<<5);  // Clear bit 5
```

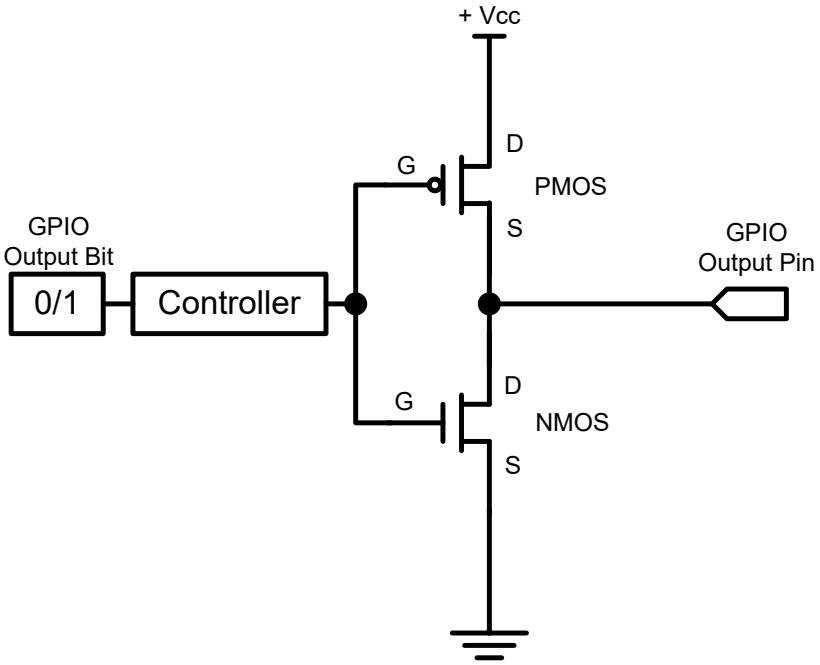
GPIO Output: Push-Pull

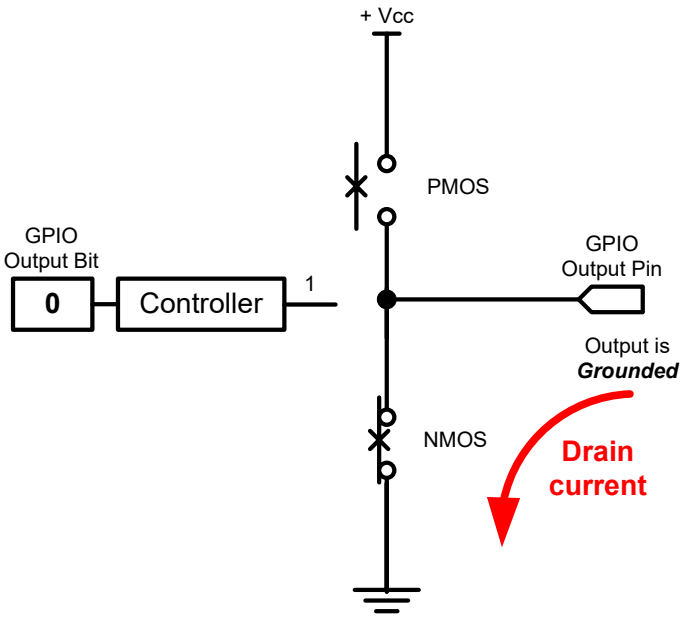




GPIO Output = 1
Source current to external circuit

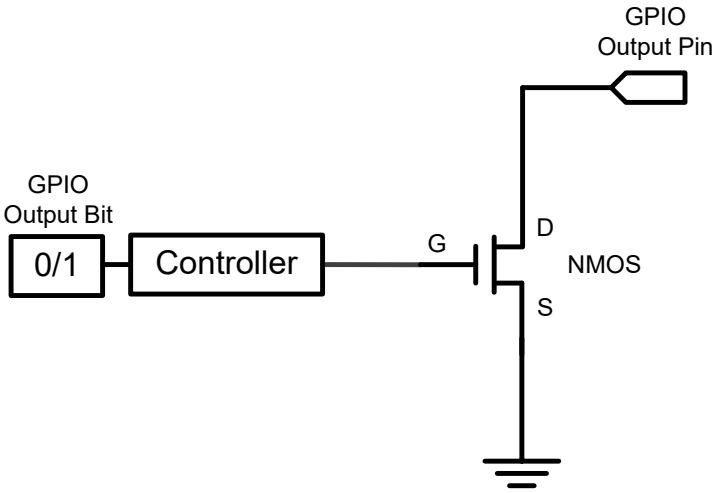
GPIO Output: Push-Pull

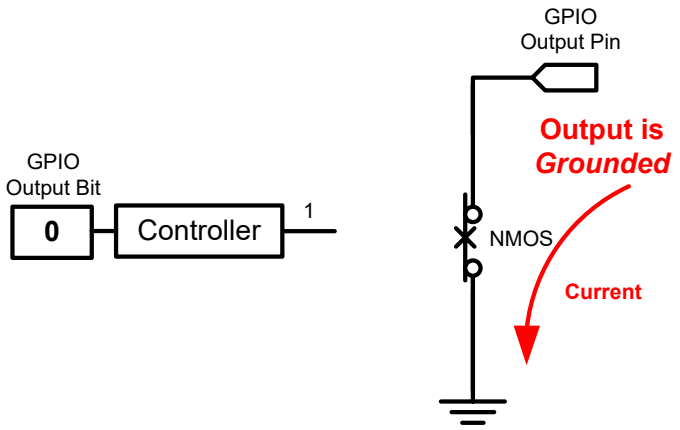




GPIO Output = 0
Drain current from external circuit

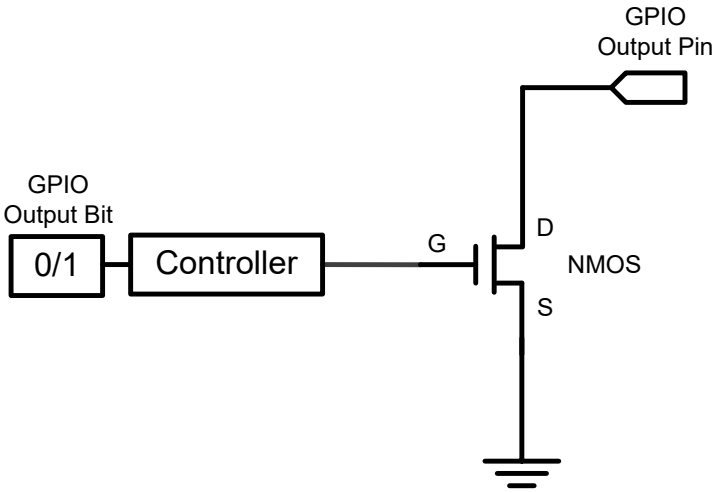
GPIO Output: Open-Drain

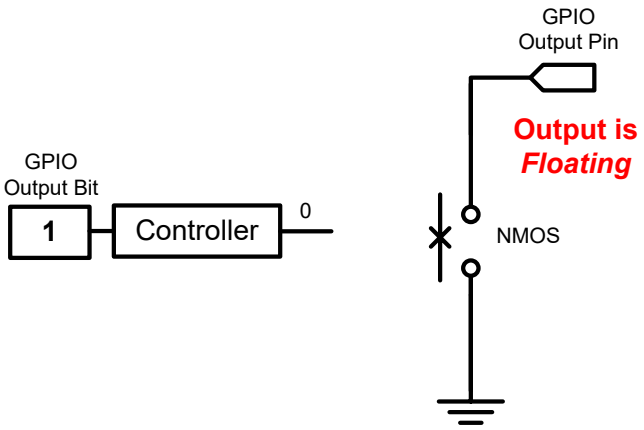




GPIO Output = 0
Drain current from external circuit

GPIO Output: Open-Drain

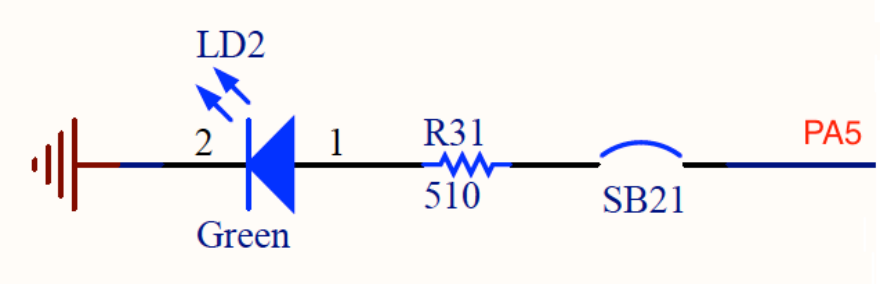




Output = 1
GPIO Pin has high-impedance to external circuit

GPIO Output: Push-Pull *vs* Open-Drain

Output Bit	Push-Pull	Open-Drain
1	High	HiZ
0	Low	Low



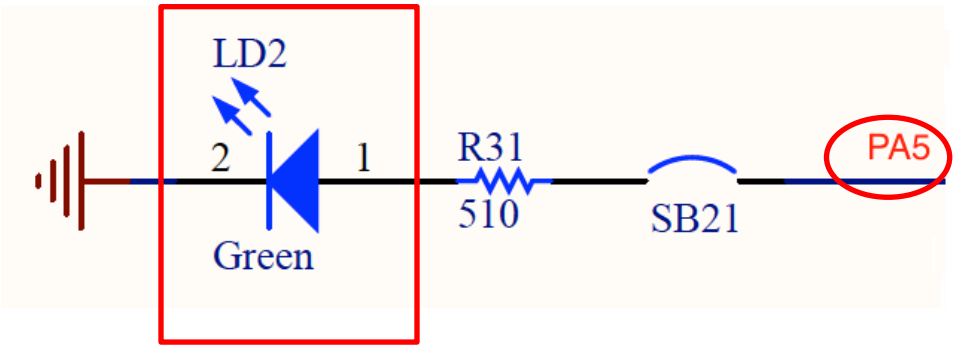
Use push-pull output, instead of open-drain output!

GPIO Output Data Register (ODR)

- 16 bits reserved, 16 data bits, 1 bit for each pin

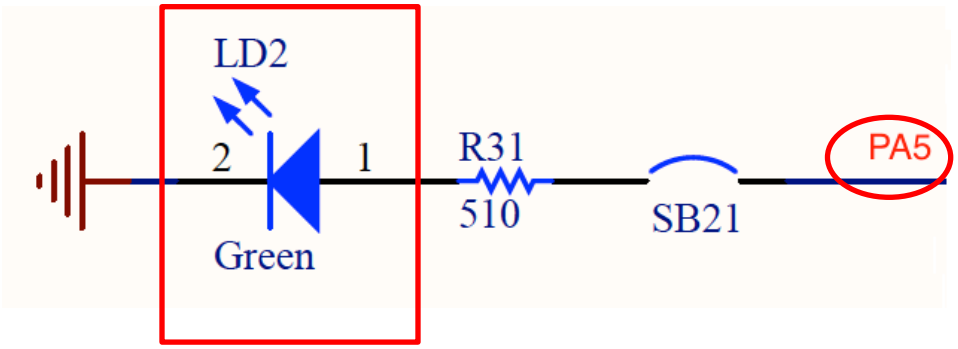
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 5




```
GPIOA->ODR |= 1UL << 5;     // Set bit 5
```

Light up the Green LED (PA.5)




```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;  // Enable clock of
Port A

GPIOA->MODER &= ~(3UL<<10); // Clear mode bits
GPIOA->MODER |=   1UL<<10;  // Set mode to output

GPIOA->OTYPE &= ~(1UL<<5); // Select push-pull output

GPIOA->ODR   |= 1UL << 5;  // Output 1 to turn on green LED
```

GPIO physical pins